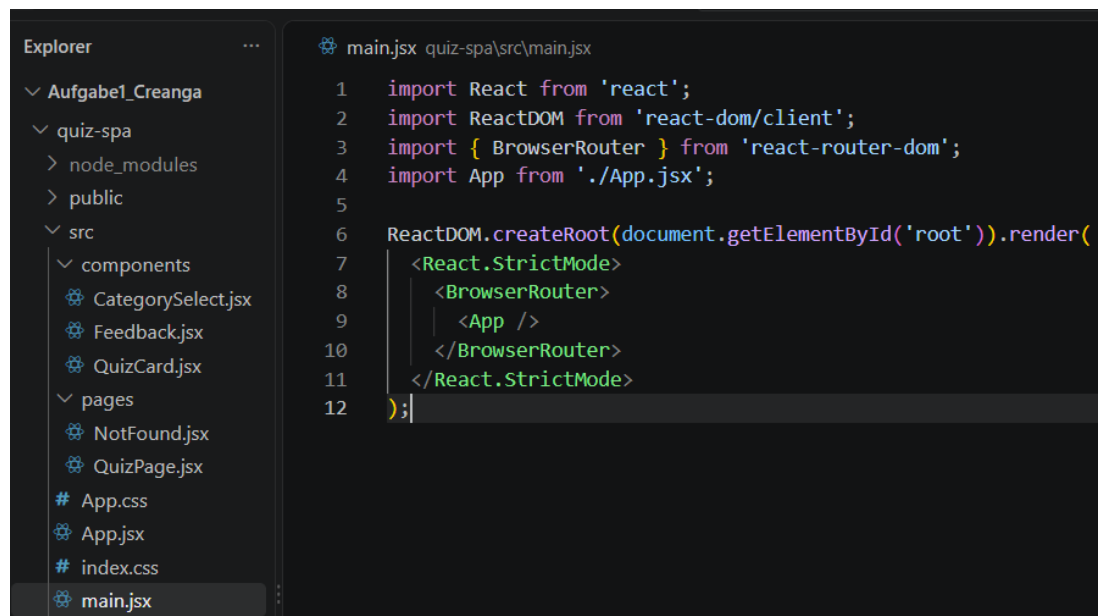
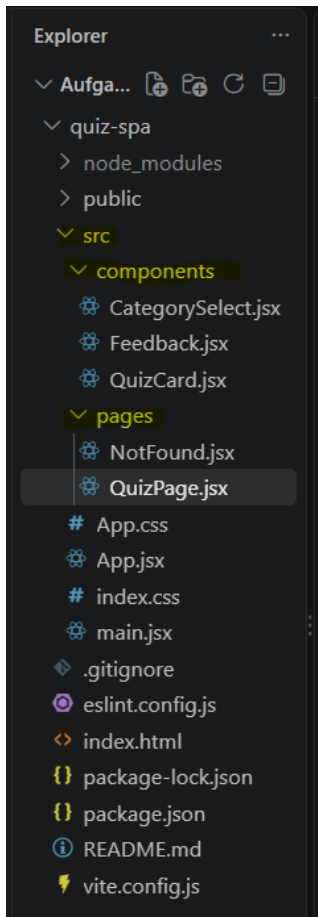


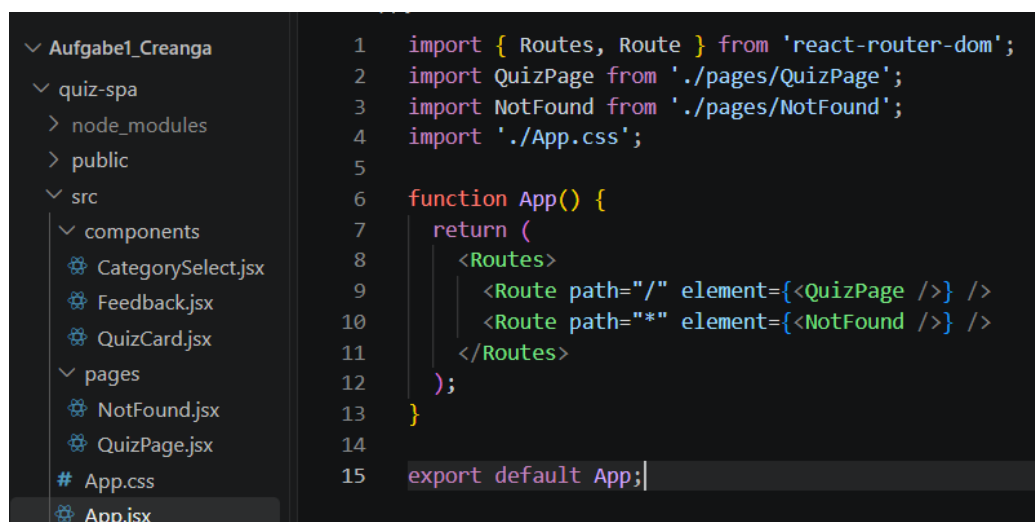
Ziel: Singel Page Application mit dem Framework React... Quiz-App

Schritt 1: Im Ordner "src" zwei Unterordner erstellen: „components“ (= für wiederverwendbare Bausteine) und „pages“ (= für die Hauptseiten). Und dann in main.jsx code rein schreiben.



- ⇒ Der erste Import importiert die React-Bibliothek selbst. (liefert Grundfunktionen.)
- ⇒ Der Zweite import, das Modul ReactDOM, das speziell für die Darstellung (das Rendern) von React-Komponenten im html-dom eines Webbrowsers zuständig ist. (stellt Funktion createRoot() zur Verfügung.)
- ⇒ Der Dritte sorgt dafür, dass die aktuelle URL im Browser ausgelesen wird und, dass React die passenden Seiten rendert, ohne die Webseite neu zu laden.
- ⇒ Der Letzte importiert meine eigene Haupt-Komponente (app.jsx) aus src.
- ⇒ Der Teil mit dem ReactDOM... sucht das div mit der ID root in der index.html und sagt react, es soll dort die komplette Anwendung hineinrendern.
- ⇒ Der Teil mit BrowserRouter aktiviert das routing von react router dom für die gesamte Anwendung.

Schritt 2: Den Code für App.jsx schreiben. Diese Datei ist das Zentrum für das Routing, sie bestimmt also welche Seite im Browser angezeigt wird, wenn der Benutzer eine bestimmte Webadresse Url aufruft.



- ⇒ Hier werden die Routing-Bausteine aus dem Paket react-router-dom geladen.
- ⇒ Die zwei paths mit den von mir erstellten Seiten, importiert beide Seiten-komponenten (Also die HauptQuizseite und die 404 Fehlerseite.) => Das * Zeichen bei dem path der Fehlerseite ist eine Wildcard („Catch-all“)
- ⇒ Main.jsx stellt also das „Fundament“ (BrowserRouter) bereit, und App.jsx nutzt dieses Fundament, um die richtigen Seiten (<Route>) auszuliefern.

Schritt 3: Die Seiten

1. QuizPage.jsx => Hauptseite. Sie holt die Fragen per API-Abruf aus dem Internet, verarbeitet die Daten (Mischen der Antworten), verwaltet den Zustand der Anwendung (also welche Kategorie ausgesucht bzw. welche Antwort angeklickt wurde) und setzt die Bausteine (= CategorySelect, QuizCard, Feedback) zusammen.

Code Erklärung:

```
QuizPage.jsx  quiz-spa\src\pages\QuizPage.jsx\...  
1  import { useState, useEffect, useCallback } from 'react';  
2  import CategorySelect from '../components/CategorySelect';  
3  import QuizCard from '../components/QuizCard';  
4  import Feedback from '../components/Feedback';  
5
```

- ⇒ Dieser Teil importiert React-Hooks und Komponente. (=> useState, useEffect, useCallback importiert die React-Hooks und drunter werden die wiederverwendbaren Unterkomponente aus dem components Ordner importiert.

```
6  function QuizPage() {  
7    const [category, setCategory] = useState('');  
8    const [questionData, setQuestionData] = useState(null);  
9    const [answers, setAnswers] = useState([]);  
10   const [selectedAnswer, setSelectedAnswer] = useState(null);  
11   const [loading, setLoading] = useState(false);  
12
```

- ⇒ Category speichert die aktuelle Kategorie ID
- ⇒ questionData hält das Fragen-Objekt, das direkt von der API zurückgeliefert wird (enthält die Frage, die richtige Antwort usw.)
- ⇒ answers: hat das fertige, gemischte Array mit allen Antworten.
- ⇒ selectAnswer speichert den Text der Antwort, auf die der Benutzer geklickt hat. (Am Anfang null).
- ⇒ Loading ist ein boolischer Wert, der anzeigt, ob gerade die Daten aus dem Internet geladen werden.

```
// Funktion zum Laden einer neuen Frage
const fetchQuestion = useCallback(async () => {
  setLoading(true);
  setSelectedAnswer(null);

  let url = 'https://opentdb.com/api.php?amount=1&difficulty=easy';
  if (category) {
    url += `&category=${category}`;
  }

  try {
    const res = await fetch(url);
    const data = await res.json();

    if (data.results && data.results.length > 0) {
      const current = data.results[0];

      // 1. Alle Antworten kombinieren
      const allAnswers = [current.correct_answer, ...current.incorrect_answers];

      // 2. Antworten zufällig mischen (Shuffle)
      const shuffled = allAnswers.sort(() => Math.random() - 0.5);

      setQuestionData(current);
      setAnswers(shuffled);
    }
  } catch (error) {
    console.error('Fehler beim Laden:', error);
  } finally {
    setLoading(false);
  }
}, [category]);
```

- ⇒ useCallback... => merkt sich die Funktion, damit sie nicht bei jedem Rendering unnötig neu erstellt wird, es sei denn, die category ändert sich.
- ⇒ Dynamische URL => Erstellt die API-Adresse und hängt optional &category=... an, falls eine Kategorie gewählt wurde.
- ⇒ Fetch(url).... Schickt eine Anfrage an die Open Trivia Database
- ⇒ Danach werden die Antworten kombiniert mithilfe des Spread-Operators, also die richtigen und falschen in ein einziges Array.
- ⇒ Der Teil mit sort math.random durchmischt dieses Array zufällig. Dadurch ist die richtige Antwort nicht immer an der gleichen Stelle.

```
useEffect(() => {
  fetchQuestion();
}, [fetchQuestion]);
```

Dieser Teil führt fetchQuestion() automatisch beim ersten Laden der Seite aus. Da fetchQuestion von category abhängt, wird automatisch eine neue Frage geladen, sobald

der Benutzer im Dropdown eine andere Kategorie auswählt.

```

1  return (
2    <div className="page-container">
3      <header className="app-header">
4        <h1> 🎯 Open Trivia Quiz</h1>
5        <CategorySelect selectedCategory={category} onSelectCategory={setCategory} />
6      </header>
7
8      <main className="quiz-main">
9        {loading ? (
10          <div className="loader">Question is loading...</div>
11        ) : questionData ? (
12          <>
13            <QuizCard
14              questionData={questionData}
15              answers={answers}
16              selectedAnswer={selectedAnswer}
17              onAnswerSelect={setSelectedAnswer}
18            />
19
20            <Feedback
21              selectedAnswer={selectedAnswer}
22              correctAnswer={questionData.correct_answer}
23            />
24
25            <button className="primary-btn reload-btn" onClick={fetchQuestion}>
26              Next question →
27            </button>
28          </>
29        ) : (
30          <p>No question found.</p>
31        )}
32      </main>
33    </div>
34  );
35 }

```

=> Wenn loading == true, wird nur der Lade Text (Question is loading...) angezeigt. Sobald die Daten da sind, werden die Bausteine gerendert.

- ⇒ CategorySelect übergibt die gewählte Kategorie und Funktion setCategory, um den State bei Änderung zu aktualisieren.
- ⇒ QuizCard übergibt die Frage, die gemischten Antworten, die aktuell gewählte Antwort und die Funktion setSelectedAnswer, wenn ein Button gedrückt wird.
- ⇒ Feedback enthält die gewählte Antwort und die richtige Antwort, um zu vergleichen und die Erfolg bzw. Fehlermeldung auszugeben.
- ⇒ Next Question Button ruft beim Klick manuell fetchQuestion auf, um eine neue Frage zu holen.

2. NotFound.jsx -Seite :

```
...  NotFound.jsx  quiz-spa\src\pages\NotFound.jsx  [🔗] default
1  import { Link } from 'react-router-dom';
2
3  function NotFound() {
4    return (
5      <div className="page-container center-content">
6        <h1>404 - Seite nicht gefunden</h1>
7        <p>Ups! Diese Adresse existiert leider nicht.</p>
8        <Link to="/" className="primary-btn">Zurück zum Quiz</Link>
9      </div>
10    );
11  }
12
13  export default NotFound;
```

- ⇒ Das ist die 404-Fehlerseite. Sie wird von der Wildcard-Route („path=“*) in der App.jsx aufgerufen, wenn der Benutzer im Browser eine Adresse eingibt, die in der App nicht existiert. Damit wird verhindert dass der Browser eine weiße oder fehlerhafte Seite anzeigt.
- ⇒ Die erste Zeile mit dem import lädt die Link Komponente aus react-router-dom. Der Browser wechselt hier die Seite, ohne die gesamte Webseite neu zu laden.
- ⇒ Die Funktion: div className ... verwendet die CSS-Klassen aus der App.css um den Inhalt in einer weißen Karte in der Mitte des Bildschirms auszurichten.
- ⇒ H1 und p zeigen dem Benutzer die Fehlermeldung
- ⇒ Link className primarybtn => erstellt einen Button, der den Benutzer beim Klick sofort zurück auf die Startseite navigiert.
- ⇒ Dann wird die Komponente in anderen Dateien verfügbar gemacht mithilfe von export default. => damit sie in App.jsx für die CatchAll Route importiert werden kann.

Schritt 4: Komponente

1. QuizCard:

```

QuizCard.jsx quiz-spa\src\components\QuizCard.jsx default
1  function QuizCard({ questionData, answers, selectedAnswer, onAnswerSelect }) {
2    return (
3      <div className="quiz-card">
4        /* HTML-Entities wie &quot; sauber darstellen */
5        <h2
6          className="question-text"
7          dangerouslySetInnerHTML={{ __html: questionData.question }}
8        />
9
10       <div className="answers-grid">
11         {answers.map((answer, index) => {
12           let btnClass = 'answer-btn';
13           if (selectedAnswer) {
14             if (answer === selectedAnswer) {
15               btnClass += answer === questionData.correct_answer ? ' selected-correct' : ' selected-incorrect';
16             } else if (answer === questionData.correct_answer) {
17               btnClass += ' show-correct';
18             }
19           }
20
21           return (
22             <button
23               key={index}
24               className={btnClass}
25               disabled={!selectedAnswer}
26               onClick={() => onAnswerSelect(answer)}
27               dangerouslySetInnerHTML={{ __html: answer }}
28             />
29           );
30         })}
31       </div>
32     </div>
33   );
34 }
35
36 export default QuizCard;

```

⇒ Für die Darstellung der aktuellen Frage und der Antwort-Buttons zuständig.

⇒ Die erste Funktion :

questionData: Das Fragen-Objekt der API (enthält z. B. question und correct_answer).

answers: Das bereits gemischte Array mit allen 4 Antwortmöglichkeiten.

selectedAnswer: Die vom Benutzer gewählte Antwort (oder null, wenn noch nichts geklickt wurde).

onAnswerSelect: Die Callback-Funktion aus QuizPage, um die gewählte Antwort im Zustand der Elternkomponente zu speichern.

⇒ **dangerouslySetInnerHTML:** Die Open Trivia API liefert Sonderzeichen in HTML-Decodierung (z. B. " anstelle von ", ' statt '). React würde diesen Text standardmäßig als einfachen Text ausgeben. Durch dangerouslySetInnerHTML werden diese Tags vom Browser richtig als Sonderzeichen gerendert.

⇒ **Logik für CSS-Klassen**

Wenn der Benutzer noch **keine** Antwort angeklickt hat (!selectedAnswer), bleibt die Klasse einfach 'answer-btn'.

Sobald eine Antwort geklickt wurde (selectedAnswer ist gesetzt):

Geklickter Button : Bekommt die Klasse selected-correct (grün), wenn er richtig war, oder selected-incorrect (rot), wenn er falsch war.

Richtige Antwort anzeigen: Falls der Benutzer die *falsche* Antwort gewählt hat, bekommt der Button mit der *richtigen* Antwort zusätzlich die Klasse show-correct, damit dem Benutzer sofort gezeigt wird, was richtig gewesen wäre.

⇒ Der return und button Teil:

- `key={index}`: Ein eindeutiger Identifier, den React beim Rendern von Listen benötigt.
- `className={btnClass}`: Wendet die dynamisch berechnete CSS-Klasse an (für das Styling von Farben und Effekten).
- `disabled={!selectedAnswer}`: Mit `!!` wird `selectedAnswer` in einen echten booleschen Wert umgewandelt (`true/false`). Sobald eine Antwort gewählt wurde, werden alle Buttons gesperrt, damit man nicht mehrfach klicken kann.
- `onClick={() => onAnswerSelect(answer)}`: Leitet die angeklickte Antwort beim Klick an die übergeordnete QuizPage weiter.
- `dangerouslySetInnerHTML={{ __html: answer }}`: Wandelt auch in den Antwort-Texten eventuelle HTML-Entities sauber in lesbare Zeichen um.

2. CategorySelect :

```
CategorySelect.jsx  quiz-spa\src\components\CategorySelect.jsx  CategorySelect  categories.map()
1  import { useEffect, useState } from 'react';
2
3  function CategorySelect({ selectedCategory, onSelectCategory }) {
4    const [categories, setCategories] = useState([]);
5
6    useEffect(() => {
7      // Kategorien aus der API laden
8      fetch('https://opentdb.com/api_category.php')
9        .then((res) => res.json())
10       .then((data) => setCategories(data.trivia_categories))
11       .catch((err) => console.error(err));
12    }, []);
13
14    return (
15      <div className="category-select">
16        <label htmlFor="category">Choose category: </label>
17        <select
18          id="category"
19          value={selectedCategory}
20          onChange={(e) => onSelectCategory(e.target.value)}
21        >
22          <option value="">All Categories (Random)</option>
23          {categories.map((cat) => (
24            <option key={cat.id} value={cat.id}>
25              {cat.name}
26            </option>
27          ))}
28        </select>
29      </div>
30    );
31  }
32
33  export default CategorySelect;
```

⇒ Dropdown-Menü zur Auswahl der Quiz-Kategorie:

- `useEffect, useState`: Werden benötigt, um die aus der API geladenen Kategorien im lokalen State zu speichern und den Abruf beim Mounten der Komponente auszuführen.

- `selectedCategory`: Die aktuell ausgewählte Kategorie-ID aus der QuizPage.
- `onSelectCategory`: Die Callback-Funktion (`setCategory`), um der QuizPage mitzuteilen, wenn der Benutzer eine neue Kategorie auswählt.

⇒ Den Zustand definieren und Kategorien per API laden

- `useState([])`: Speichert die von der API gelieferte Liste von Kategorien als Array.
- `useEffect(..., [])`: Führt den `fetch()`-Aufruf genau einmal aus, wenn die Komponente geladen wird (leeres Dependency-Array []).
- `fetch(['https://opentdb.com/api_category.php'])(https://opentdb.com/api_category.php')`: Ruft den Endpunkt der Open Trivia API ab, der alle verfügbaren Themenbereiche (z. B. Film, Musik, Science) zurückliefert.
- `setCategories(data.trivia_categories)`: Speichert das gefundene Array von Kategorien im State `categories`.

⇒ Das html Dropdown Menü:

- `<label>` & `<select>`: Verbindet das Textlabel barrierefrei mit dem Auswahlfeld über die `id="category"` und `htmlFor="category"`.
- `value={selectedCategory}`: Macht das Dropdown zu einer sogenannten Controlled Component (der angezeigte Wert entspricht immer dem Zustand in React).
- `onChange={(e) => onSelectCategory(e.target.value)}`: Sobald der Benutzer eine Option auswählt, wird der neue Wert (`e.target.value`, also die ID der Kategorie) direkt an QuizPage übermittelt.
- `<option value="">All Categories (Random)</option>`: Die Standard-Option. Ein leerer String "" bedeutet, dass keine spezielle Kategorie gewählt ist und Fragen aus allen Themenbereichen zufällig geladen werden.
- `categories.map(...)`: Iteriert über das im State gespeicherte Array von Kategorien und generiert für jede Kategorie dynamisch ein `<option>`-Element mit der `cat.id` als Wert und dem `cat.name` als angezeigtem Text.

3. Feedback :

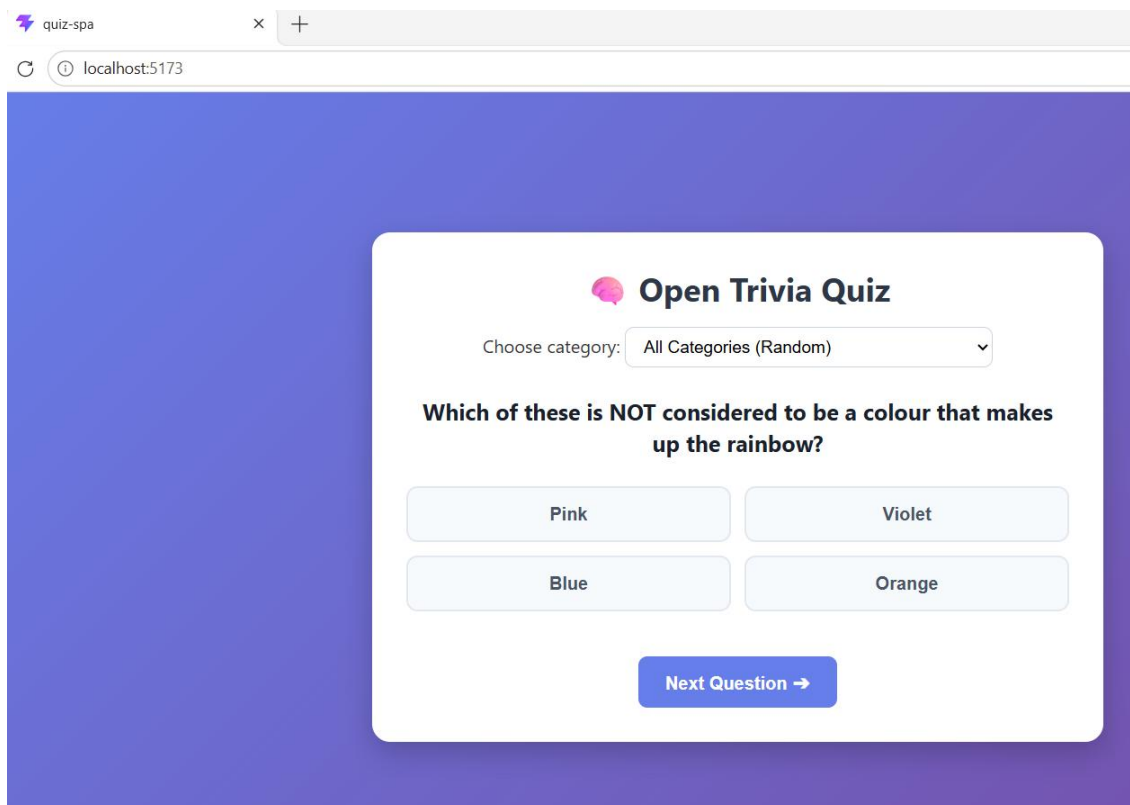
```
Feedback.jsx quiz-spa/src/components/Feedback.jsx Feedback
1 function Feedback({ selectedAnswer, correctAnswer }) {
2   if (!selectedAnswer) return null;
3
4   const isCorrect = selectedAnswer === correctAnswer;
5
6   return (
7     <div className={`feedback ${isCorrect ? 'correct' : 'incorrect'}>
8       {isCorrect ? (
9         <p>🎉 Right! Great job!</p>
10      ) : (
11        <p>
12          ❌ Wrong! The correct answer is: { ' ' }
13          <strong dangerouslySetInnerHTML={{ __html: correctAnswer }} />
14        </p>
15      )}
16     </div>
17   );
18 }
19
20 export default Feedback;
```

Ist für die textuelle Rückmeldung nach einer Antwortauswahl zuständig. Sie prüft, ob die wählte Antwort richtig oder falsch war, und gibt ein passendes Feedback samt Einfärbung aus. Bei einer falschen Antwort blendet sie auch die korrekte Lösung ein.

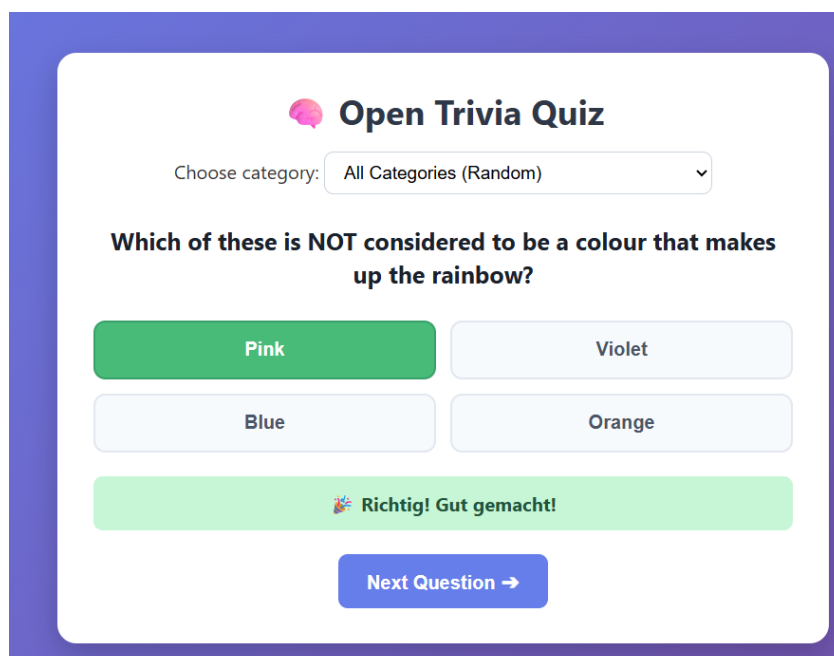
- **selectedAnswer & correctAnswer:** Die Komponente erhält die vom Benutzer geklickte Antwort und die tatsächliche Lösung aus der QuizPage übergeben.
- **if (!selectedAnswer) return null;** Solange der Benutzer noch keine Antwort gewählt hat, gibt die Komponente null zurück. Dadurch wird sie im DOM gar nicht erst gerendert und bleibt komplett unsichtbar.
- **isCorrect:** Ein boolescher Vergleich (true/false), der prüft, ob die gewählte Antwort exakt mit der korrekten Lösung übereinstimmt.

Schritt 5: App.css: Styling

Screenshots:



Richtige Antwort :



Falsche Antwort :

 **Open Trivia Quiz**

Choose category: All Categories (Random) ▼

Who is the main antagonist in the Portal franchise?

GLaDOS

Rick

Chell

Wheatley

✖ Falsch! Die richtige Antwort ist: **GLaDOS**

Next Question →